

SupKonQuest — Documentation technique

Document destiné aux développeurs. Dernière mise à jour : 2026-05-31.

Table des matières

1. Architecture et choix techniques

- 1.1 Vue d'ensemble
- 1.2 Godot 4.5
- 1.3 C# (.NET 8.0)
- 1.4 Cartes en presets RLE
- 1.5 Nakama (multijoueur en ligne)

2. Structures de données

- 2.1 Économie et équipes — GameManager
- 2.2 Camps — CampSimple
- 2.3 Unités — machine à états
- 2.4 Territoire — graphe de régions
- 2.5 Réseau — registre d'entités
- 2.6 Session — GameState

3. IA et algorithmique

- 3.1 Pathfinding
- 3.2 IA — Utility AI
- 3.3 Formules de combat

4. Mécaniques et game design

- 4.1 Capture de camp
- 4.2 Tiers de production
- 4.3 Ports
- 4.4 Transport
- 4.5 Victoire
- 4.6 Camps neutres
- 4.7 Bonus vitesse

5. Réseau

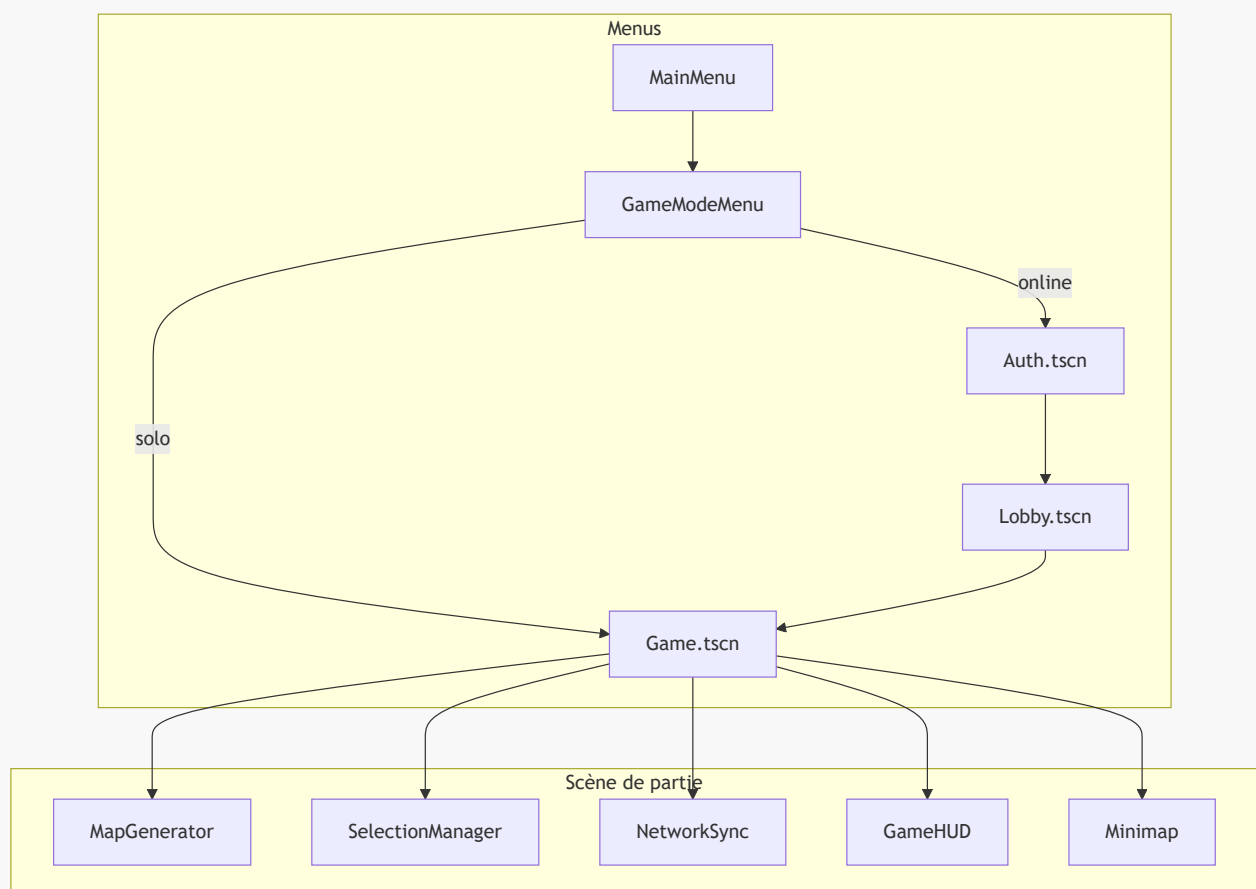
- 5.1 Parcours multijoueur
- 5.2 Module relay supkonquest_relay
- 5.3 Matchmaking
- 5.4 Opcodes lobby
- 5.5 Opcodes gameplay relay
- 5.6 Opcodes déconnexion et reconnexion
- 5.7 Enveloppe et validation serveur

- [5.8 Déterminisme et simulation locale](#)
6. [Écarts au cahier des charges](#)
 - [6.1 Partiellement implémentées](#)
 - [6.2 Manquantes](#)
 - [6.3 Écarts assumés comme conformes](#)
 7. [Module reference \(extrait\)](#)
 8. [Parcours de lecture recommandé](#)
 9. [Annexe — Serveur Nakama local \(développeurs\)](#)
-

1. Architecture et choix techniques

1.1 Vue d'ensemble

SupKonQuest est un RTS 2D de conquête territoriale. Le joueur contrôle des camps, produit des unités terrestres et navales, et cherche à dominer la carte. Le code est organisé en **scènes Godot réutilisables**, complétées par des **modules globaux** (chargés une fois au démarrage) et des **fichiers partiels** qui découpent les grosses classes ([Unit](#), [Ship](#), [CampSimple](#)) par sujet.



Les informations partagées entre les menus et la partie (or, langue, session en ligne...) vivent dans des **modules globaux** : `GameManager`, `GameState`, `NakamaService`, `LocalizationManager`, `KeybindingsManager`, `AudioSettings`.

1.2 Godot 4.5

Pourquoi Godot ?

- Moteur open-source, léger, bien adapté au 2D (tuiles, navigation, signaux).
- Chaque élément du jeu (unité, camp, navire) est une scène réutilisable (`Unit.tscn`, `camp_simple.tscn`, `Ship.tscn`).
- Export Windows / macOS / Linux vers un exécutable autonome.
- Intégration C# native via .NET 8.

Mécanismes Godot utilisés :

Mécanisme	Rôle dans le projet
<code>NavigationAgent2D</code>	Calcul de trajet à pied et en mer (deux cartes de navigation distinctes)
Groupe(s) (<code>camps</code> , <code>units</code> , <code>ships</code>)	Retrouver rapidement toutes les unités ou tous les camps sans parcourir l'arbre de scène
Signaux	Avertir l'interface quand un joueur rejoint, qu'une partie démarre, etc.
Fichiers partiels (<code>partial class</code>)	Découper <code>Unit</code> , <code>Ship</code> , <code>CampSimple</code> en combat, mouvement, production...
Modules globaux (autoloads)	Conserver l'or, la langue et l'état de session entre les changements de scène

1.3 C# (.NET 8.0)

Pourquoi C# plutôt que GDScript ?

Critère	Apport concret pour SupKonQuest
Typage statique	Le compilateur détecte les erreurs de type avant l'exécution (mauvais paramètre, propriété inexistante). Sur ~50 scripts et des structures imbriquées (files de production, graphe de régions, registre réseau), cela réduit les régressions.
Standard industrie	C# et .NET sont largement utilisés en développement de jeux (Unity) et en entreprise. Compétences transférables, documentation abondante, outillage mature (Rider, Visual Studio, analyse statique).
Performance	.NET 8 compile en code natif. Les boucles de gameplay (<code>_Process</code> , IA, pathfinding) et les collections (<code>Dictionary</code> , <code>Queue</code> , <code>HashSet</code>) bénéficient d'un runtime optimisé, adapté à une montée en charge d'unités en fin de partie.

GDScript aurait été plus rapide à prototyper pour un très petit projet, mais le volume de logique métier (économie, IA, réseau, capture) justifie un langage typé et outillé pour la maintenance.

Conventions de code : PascalCase pour les types et méthodes, `_camelCase` pour les champs privés.

1.4 Cartes en presets RLE (pas de génération procédurale)

Les cartes sont stockées sous forme **compacte** (Run-Length Encoding : une même tuile répétée n'est encodée qu'une fois) dans des classes statiques (`IrridiumMap`, `AlabastaMap`, `TorskeyMap`). Au chargement, le générateur décode ces données, peint la carte, calcule les zones praticables et place les camps aux positions prévues.

Justification :

- **Même carte pour tous les joueurs en ligne** : avec la même graine (`MapSeed`), tout le monde obtient la même disposition — indispensable en multijoueur.
- **Cartes équilibrées à la main** : régions, côtes et emplacements de camps sont réfléchis, pas tirés au hasard.
- **Maintenance simple** : pas de générateur procédural complexe à déboguer.

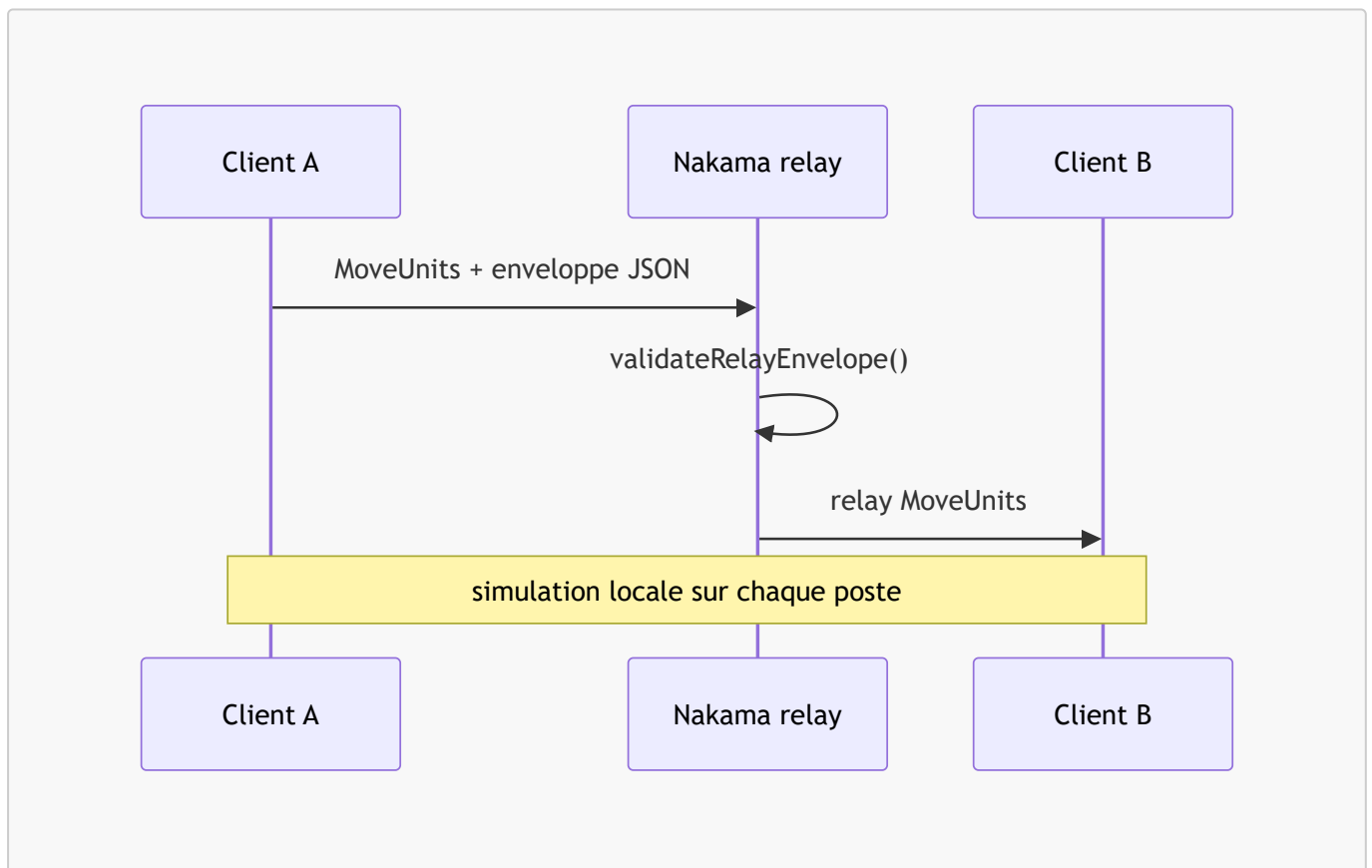
Le shuffle des camps utilise Fisher-Yates avec une seed partagée (`GameManager.AssignCampsToPlayers()`).

1.5 Nakama (multijoueur en ligne)

Pourquoi Nakama ?

- Serveur open-source dédié au jeu en ligne : comptes, matchmaking, connexion WebSocket.
- Module TypeScript personnalisable pour le lobby et le relais des commandes de jeu.
- Le mode solo et le mode en ligne restent séparés via `GameState.IsOnline`.

Modèle retenu : relais de commandes (chaque joueur fait tourner le jeu chez lui). Quand vous déplacez une unité, votre client envoie l'ordre au serveur, qui le transmet aux autres. Le serveur vérifie que le message est bien formé (identité, numéro de séquence, format JSON) mais ne recalcule pas toute la partie.



Pourquoi pas un serveur autoritaire ? Moins complexe pour un RTS, réutilise la logique solo. Inconvénient accepté : risque de divergence en cas de triche ou bug ; reconnexion mid-game non finalisée.

2. Structures de données

2.1 Économie et équipes — GameManager

```
Dictionary<int, int> _teamGold;           // or par équipe (teamId → gold)
Dictionary<int, int> _homeRegions;       // région d'origine par équipe
HashSet<int> _tier2Unlocked;             // équipes ayant acheté le tier 2
Dictionary<int, float> _speedMultipliers; // bonus vitesse par région contrôlée
List<CampSimple> _allCamps;              // registre de tous les camps
```

Constantes économiques :

Constante	Valeur
StartingGold	100
PassiveGoldPerSecond	75 (par équipe, joueur et IA)
GoldPerSecond (par camp)	50
RegionBonusGold	30 (région entière contrôlée)
CaptureBonus	50 (instantané à la capture)
MaxGold	9999

Parité joueur / IA : il n'existe pas de taux d'or séparé pour les bots. En solo, chaque équipe reçoit le passif global ; le débit total par seconde vaut :

$$G_{/s} = 75 + 50 \times n_{\text{camps}} + 30 \times n_{\text{régions complètes}}$$

Les captures ajoutent +50 or instantané — pour toutes les équipes, y compris celles pilotées par **AIController**.

2.2 Camps — CampSimple

```
List<Unit> _spawnedUnits;                // toutes les unités liées au camp
List<Unit> _defenders;                   // défenseurs (initiaux + bonus capture)
Queue<string> _productionQueue;         // file unités (max 7)
Queue<string> _shipProductionQueue;     // file navires (max 5)
List<Ship> _spawnedShips;               // navires actifs du port
```

Propriétés exportées clés : `TeamId`, `IsNeutralCamp`, `MaxHealth` (600), `RegionId`, `CampId`.

Défenseurs neutres spawnés : `["Infantry", "Support", "Heal", "Range"]` avec `IsNeutralCampUnit = true` (HP × 1,5).

2.3 Unités — machine à états

```
enum UnitState {
    Idle, MovingToTarget, Attacking, MovingToPoint,
    Healing, MovingToTransport, AttackingCamp
}
```

Stats centralisées dans `UnitStats` (`Dictionary<string, UnitStatsData>`). Navires dans `ShipStats`.

2.4 Territoire — graphe de régions

`TerritoryConnectivity` construit un **graphe non orienté** :

```
Dictionary<int, HashSet<int>> // regionId → voisins adjacents terrestres
```

Construction : parcours 4-voisinage sur la grille de territoire ; arête si deux tuiles terrestres adjacentes appartiennent à des régions différentes (> 0).

Usage : l'IA et le joueur priorisent les camps atteignables à pied ; les cibles isolées par l'eau passent par le naval.

2.5 Réseau — registre d'entités

En multijoueur, les joueurs doivent parler de **la même unité, le même camp, le même navire**. Chaque entité reçoit un identifiant réseau (`NetworkId`) : une chaîne de caractères stable, partagée entre toutes les machines.

`NetworkEntityRegistry` est l'**annuaire** qui fait le lien entre cet identifiant et l'objet Godot correspondant (noeud `Unit`, `Ship` ou `CampSimple`).

```
// NetworkEntityRegistry.cs — structure centrale
private static readonly Dictionary<string, Node> _entities = new();

public static void Register(string networkId, Node node)
    => _entities[networkId] = node;

public static T Get<T>(string networkId) where T : Node
{
    if (_entities.TryGetValue(networkId, out var node) && node is T typed)
        return typed;
    return null;
}
```

Deux stratégies d'identifiants :

Type	Format	Usage
Déterministe	<code>camp_{campId}_unit_{index}</code>	Défenseurs initiaux d'un camp : même ID sur toutes les machines → même spawn, pas besoin d'un message réseau par défenseur
Dynamique	<code>{peerId}_{counter}</code>	Unités et navires produits en cours de partie : ID unique généré à la création

Cycle de vie :

1. À la création d'une entité réseau, `Register(networkId, node)` l'ajoute à l'annuaire.
2. Quand un message arrive (« l'unité X subit Y dégâts »), le routeur réseau appelle `Get<Unit>(networkId)` pour trouver la cible locale.
3. À la mort ou à la fin de partie, `Unregister` ou `Clear` nettoie l'annuaire.

Sans ce registre, chaque message réseau devrait transporter des coordonnées ou des références fragiles ; avec lui, un simple identifiant suffit pour appliquer dégâts, déplacements et captures de façon cohérente.

2.6 Session — GameState

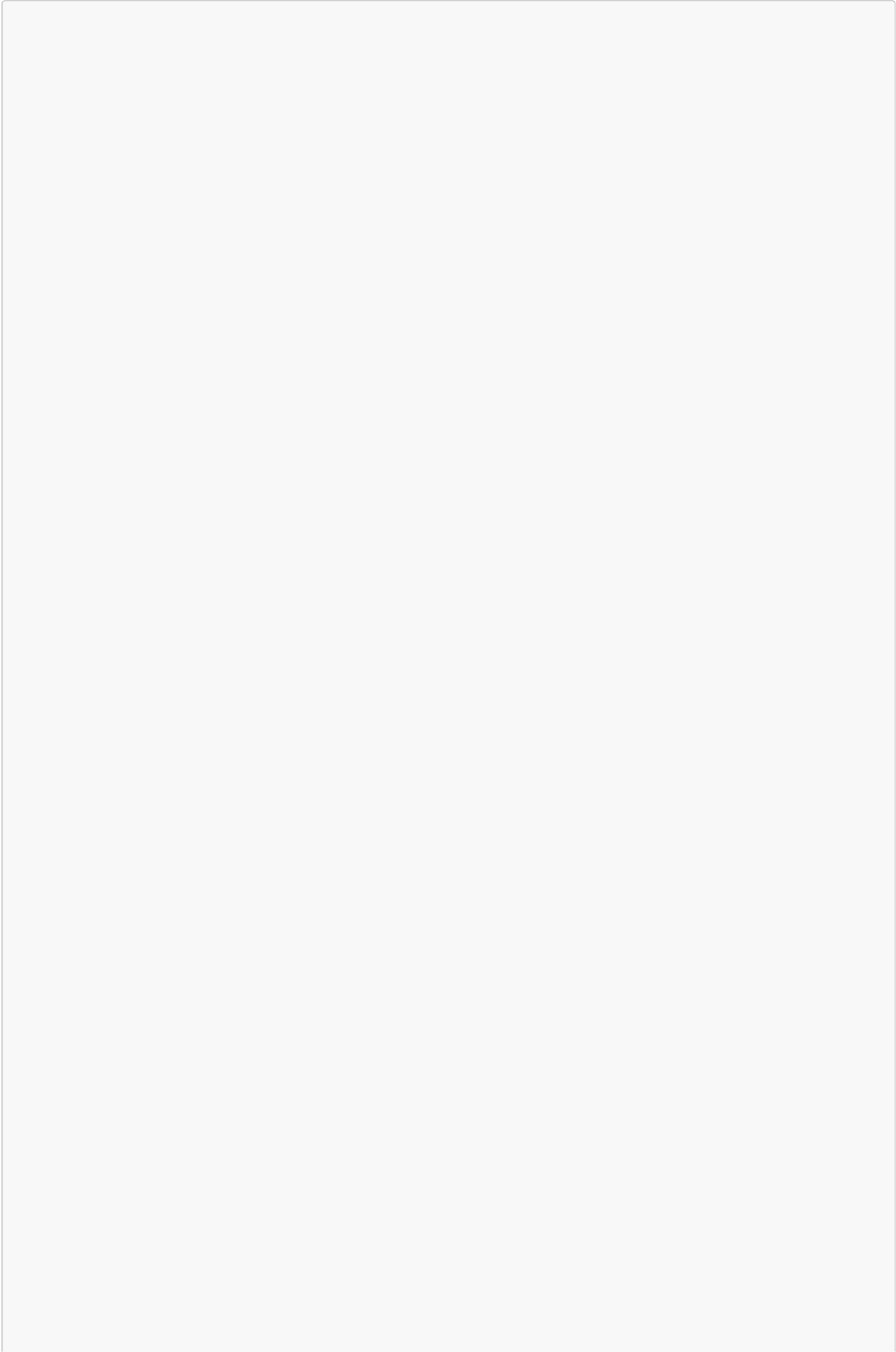
GameState est le **carnet de bord de la partie en cours** : il répond à « on joue comment, où, avec qui ? ». C'est un module global persistant : il survit au passage du menu à la scène de jeu et centralise les paramètres que les autres scripts consultent.

Données principales :

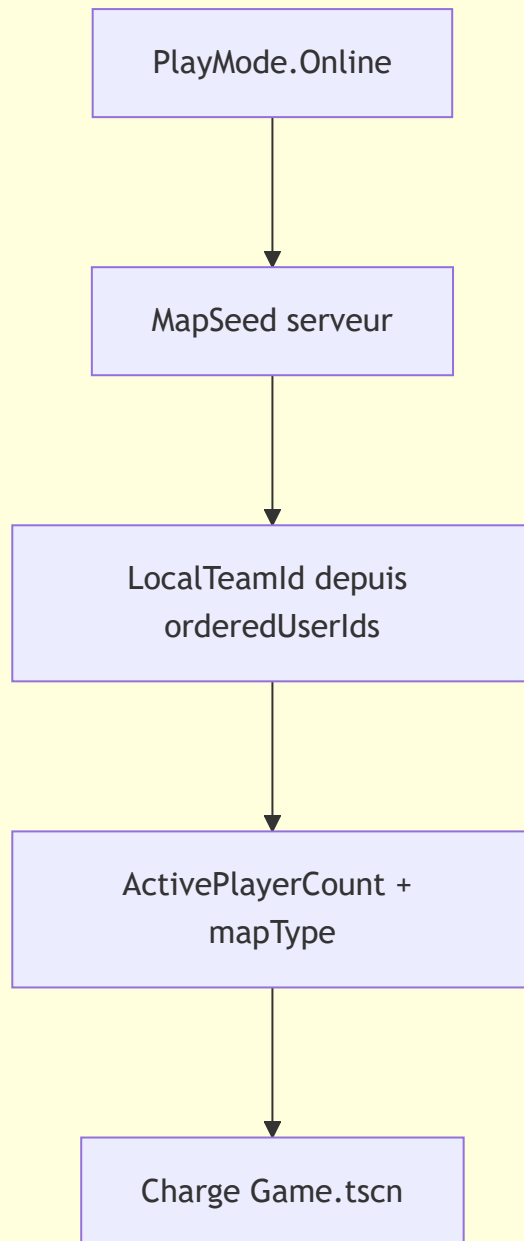
```
public enum PlayMode { Offline, Online }
public enum MapType { Irridium, Alabasta, Torskey }

public PlayMode CurrentPlayMode; // solo ou en ligne
public int MapSeed; // graine pour la carte identique partout
public int LocalTeamId; // numéro d'équipe du joueur local (1, 2, 3...)
public int ActivePlayerCount; // nombre de joueurs humains (1 solo, 2-8 en ligne)
public bool IsAIMode; // IA activée (solo uniquement)
public MapType SelectedMapType; // carte choisie ou tirée au sort
public bool FastMode; // accélération du temps (solo)
```

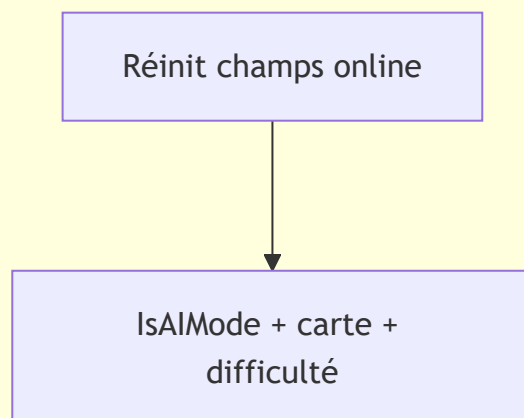
Deux chemins de démarrage typiques :

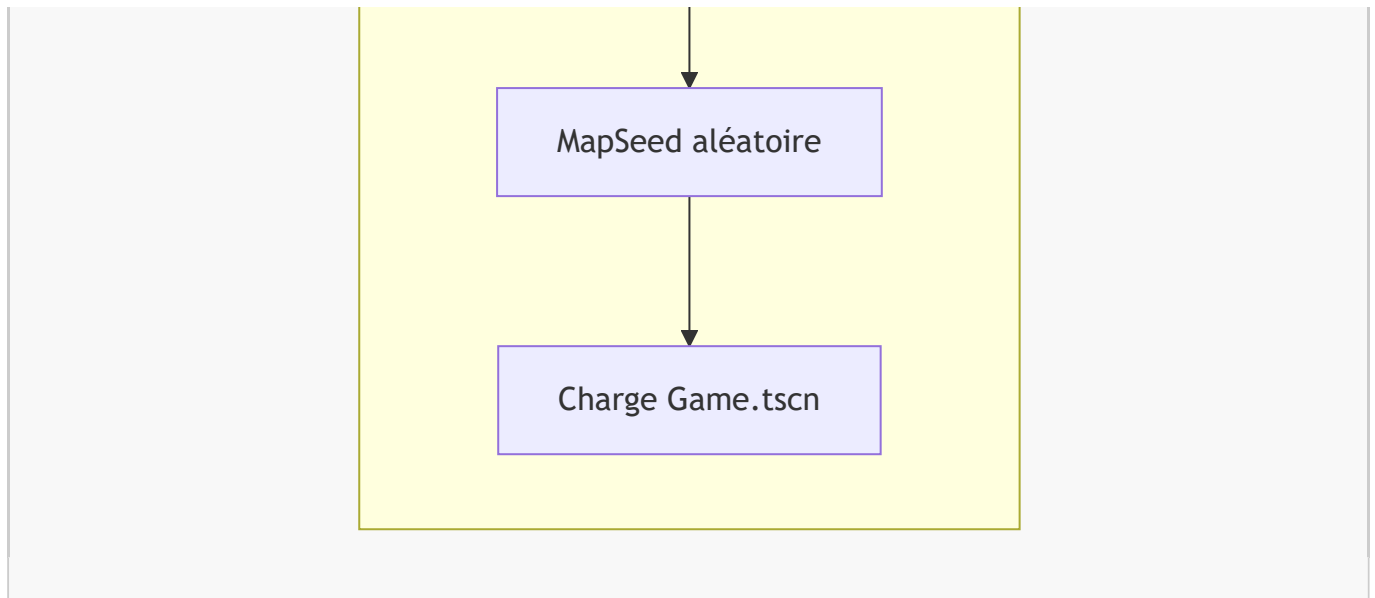


En ligne – ConfigureOnlineMatch()



Solo – StartSoloGame()





Pourquoi un objet dédié plutôt que de tout mettre dans `GameManager` ?

- **Séparation des rôles** : `GameManager` gère l'or et les règles ; `GameState` gère le contexte de lancement (mode, seed, équipe locale).
- **Ordre d'initialisation** : la scène `Game.tscn` peut être recrée à chaque partie alors que `GameState` reste en mémoire — pratique pour revenir au menu sans perdre la config en cours.
- **Tests et lecture** : un seul endroit pour savoir si on est en solo, en ligne, avec quelle seed et quelle carte.

3. IA et algorithmique

3.1 Pathfinding

Les unités et navires ne se déplacent pas en ligne droite aveugle : elles suivent une **carte de navigation** (zones praticables calculées au chargement de la carte).

- **À pied** : tuiles sable, herbe, forêt (`MapGenerator.BuildNavigationMesh()`).
- **En mer** : tuiles d'eau uniquement (`BuildWaterNavigationMesh()`).
- **Blocage** : si la vitesse réelle reste inférieure à 10 % de la vitesse attendue pendant 2 secondes, l'unité abandonne et repasse en attente.

Extrait simplifié du déplacement (`Unit.Movement.cs`) :

```
private void MoveWithNav(Vector2 targetPos)
{
    float moveSpeed = _stats.Speed * (GameManager.Instance?.GetSpeedMultiplier(TeamId) ?? 1f);

    // Recalcul du chemin seulement si la destination a assez bougé
    if (_navTargetDirty || targetPos.DistanceTo(_lastNavTargetPos) > NavUpdateDistance)
    {
        _navAgent.TargetPosition = targetPos;
        _lastNavTargetPos = targetPos;
        _navTargetDirty = false;
    }
}
```

```

    if (_navAgent.IsNavigationFinished()) return;

    Vector2 nextPos = _navAgent.GetNextPathPosition();
    Vector2 direction = (nextPos - GlobalPosition).Normalized();
    ApplyMovementVelocity(direction * moveSpeed);
}

```

L'agent recalcule le chemin seulement si la destination a suffisamment bougé (**NavUpdateDistance**), pour limiter le coût processeur quand des dizaines d'unités se déplacent en même temps.

3.2 IA — Utility AI (**AIController**)

L'IA adverse ne suit pas un script fixe du début à la fin. Toutes les quelques secondes (selon la difficulté), elle **évalue plusieurs options** — produire, attaquer, défendre, agir en mer — et choisit celle qui semble la plus utile. C'est le principe de l'**Utility AI** : noter chaque possibilité, prendre la meilleure.

Boucle principale (**AIController._Process**) :

```

_tickTimer -= dt;
if (_tickTimer > 0f) return;
_tickTimer = TickInterval[_diffIdx]; // 6 s / 3,5 s / 2 s selon Easy/Medium/Hard

RunTick(); // production + combat + éventuellement naval

```

Production : l'IA achète le tier 2 dès qu'elle a 1500 or (Medium/Hard), puis produit selon une composition cible :

```

// Easy : 100 % Infantry
// Medium : 45 % Infantry, 35 % Range, 20 % Support (+ Heal/AntiArmor au tier 2)
private static readonly Dictionary<string, float>[] TargetComposition = { ... };

```

Choix de cible : chaque camp ennemi reçoit un score ; le plus haut est attaqué en priorité :

$$\text{score} = 2500 \cdot 1_{\text{neutre}} + 3500 \cdot 1_{\text{home}} - 0,4 \times d$$

où d est la distance entre le centre de masse de l'IA et le camp, $1_{\text{neutre}} = 1$ si le camp est neutre, et $1_{\text{home}} = 1$ si le tier 3 n'est pas débloqué et que le camp est dans la région d'origine.

```

private float ScoreCamp(CampSimple camp, int currentTier, int homeRegion)
{
    float score = 0f;
    if (camp.IsNeutralCamp) score += 2500f;
    if (currentTier < 3 && camp.RegionId == homeRegion) score += 3500f;
    score -= aiCenter.DistanceTo(camp.GlobalPosition) * 0.4f;
    return score;
}

```

En **Facile**, l'IA ignore ces bonus et attaque simplement le camp le plus proche. En **Difficile**, elle vise la région d'origine, adapte sa production (plus d'AntiArmor si l'ennemi produit beaucoup de Heavy) et peut lancer des vagues amphibies.

Trois niveaux — paramètres clés :

Paramètre	Easy	Medium	Hard
Intervalle de décision	6 s	3,5 s	2 s
Max unités	8	16	28
Délai 1re attaque	20 s	12 s	5 s
Taux d'erreur (mauvaise cible)	40 %	15 %	0 %
Ratio défense	0 %	15 %	20 %

Ciblage terrestre : l'IA ignore les camps dont la région n'est pas reliée à la sienne par la terre (`TerritoryConnectivity.AreConnected()`). Les îles ou zones séparées par l'eau passent par le naval.

3.3 Formules de combat

Dégâts après armure (formule du cahier des charges) :

$$d_{\text{réels}} = \frac{d_{\text{attaque}} \times 100}{100 + d_{\text{défense}}}$$

La défense totale inclut le bonus des unités Support proches : $d_{\text{défense}} = d_{\text{base}} + b_{\text{support}}$.

Implémentation (`Unit.Combat.cs`) :

```
public void TakeDamageFrom(float damage, int attackerTeamId)
{
    float totalDefense = _stats.Defense + GetSupportDefenseBonus();
    float actualDamage = damage * 100f / (100f + totalDefense);
    _currentHealth -= actualDamage;
    if (_currentHealth <= 0) Die();
}
```

AntiArmor vs Heavy : $d_{\text{réels}} \leftarrow 2 \times d_{\text{réels}}$.

Mortar — dégâts de zone avec atténuation :

$$d_{\text{splash}} = d_{\text{max}} \times \left(1 - \frac{\text{distance}}{r_{\text{splash}}} \right)$$

Rayon $r_{\text{splash}} = 200$ px, $d_{\text{max}} = 20$ au centre, sans tenir compte de l'armure.

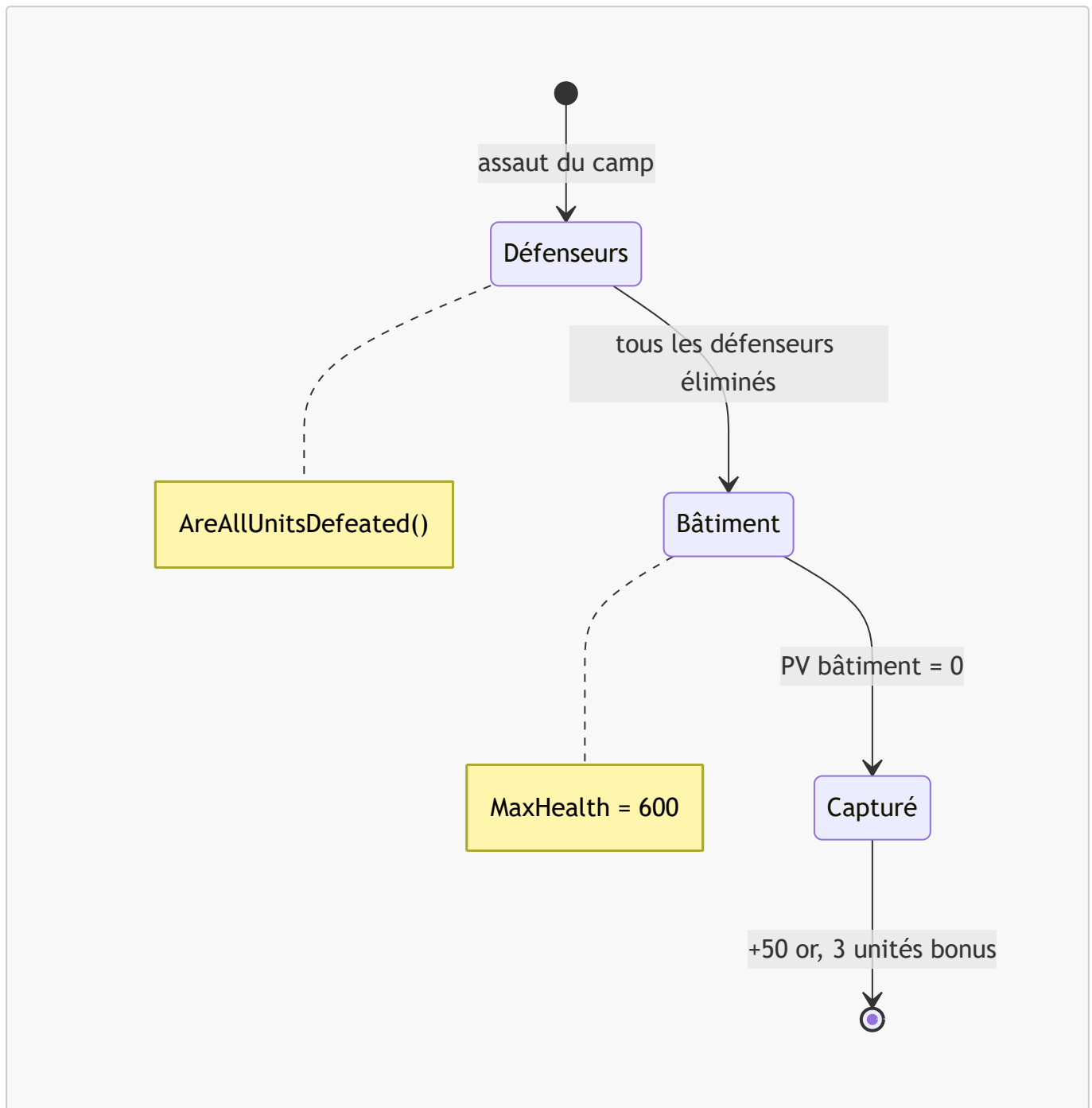
Support — aura passive : +10 défense par Support allié dans 200 px (cumul plafonné).

Ultimes d'équipe (soin / bonus défense, activables par le joueur) :

Ultime	Portée	Effet	Recharge
Heal	300 px	+70 PV en zone	20 s
Support	320 px	+20 défense, 8 s	25 s

4. Mécaniques et game design

4.1 Capture de camp (deux phases — choix assumé)

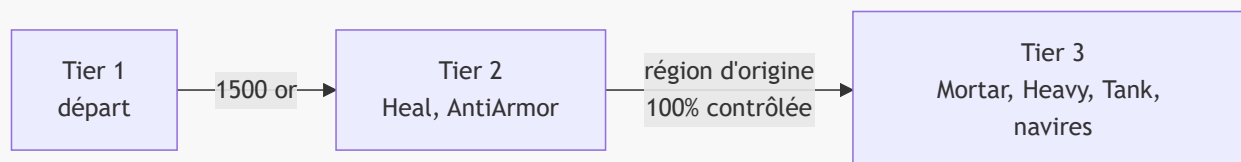


Récompenses : +50 or, 3 unités bonus (Infantry, Range, Infantry), or stocké dans le camp neutre transféré.

Justification : gameplay plus tactique qu'une capture instantanée au dernier défenseur ; fenêtre de contre-attaque possible pour celui qui se fait attaquer.

4.2 Tiers de production

Les unités avancées ne sont pas obtenues en capturant des camps neutres « premium ». Elles passent par un **système d'amélioration volontaire** : le joueur investit son or et contrôle du territoire pour monter en puissance.



Tier	Condition	Unités / navires
1	Départ	Infantry, Support, Range
2	Achat 1500 or (bouton HUD)	+ Heal, AntiArmor
3	Tous les camps de votre région d'origine	+ Mortar, Heavy, Tank, Transport, Frégate, Destroyer

Pourquoi un achat tier 2 plutôt que des unités avancées chez les neutres ?

- **Gestion de l'économie** : 1500 or représente un investissement notable (environ 20s de revenus au début de partie avec un camp, plus si l'on produit des unités entre-temps). Le joueur doit choisir entre produire massivement des unités bon marché ou **épargner** pour débloquent Heal et AntiArmor — dilemme stratégique central du mid-game.
- **Pacing maîtrisé** : si les camps neutres spawnaient Heavy ou Mortar, une early agression deviendrait un mur imprévisible. L'upgrade tier 2 récompense la planification, pas le hasard du premier camp capturé.
- **Lisibilité** : l'adversaire sait qu'un tier 2 annonce des soins et du anti-blindé ; la progression reste visible (bouton HUD, composition d'armée).

Pourquoi le tier 3 lié à la région d'origine ?

- **Objectif territorial** avant la puissance de feu lourde et la marine : il faut **consolider sa zone de départ** avant d'accéder au Mortar, au Tank et aux navires.
- **Synergie avec les bonus région** : contrôler sa région entière donne déjà +30 or/s ; le tier 3 récompense la même ambition (tenir tout un secteur).
- **Contre-poids naval** : le port et les navires (tier 3) arrivent une fois la base sécurisée, évitant une course navale dès les premières minutes.

Implémentation (`GameManager.GetUnlockedTier`) :

```
public int GetUnlockedTier(int teamId)
{
    if (!_tier2Unlocked.Contains(teamId)) return 1;

    if (!_homeRegions.TryGetValue(teamId, out int homeRegion)) return 2;
```

```

var homeCamps = _allCamps.FindAll(c => c.RegionId == homeRegion);
if (homeCamps.TrueForAll(c => c.GetTeamId() == teamId)) return 3;

return 2;
}

```

Limite navires : 5 actifs par équipe (`ShipStats.MaxActiveShipsPerTeam`).

4.3 Ports

- Achat manuel : 500 or via HUD.
- Placement : tuile terrestre du territoire du joueur, adjacente à l'eau.
- Orientation auto selon direction de l'eau adjacente.
- Annulation rembourse l'or.

4.4 Transport

Capacité : 10 unités. Embarquement : clic droit sur transport allié. Débarquement : clic droit sur côte.

4.5 Victoire

`VictoryManager.CheckVictoryCondition()` : une équipe contrôle **100 % des camps non neutres**.
Vérification chaque seconde.

4.6 Camps neutres

- Pas de production, pas de génération d'or, pas de tourelle active.
- Défenseurs réactifs uniquement si troupes ennemies entrent dans le rayon territorial (600 px).
- 4 défenseurs initiaux (Infantry, Support, Heal, Range) avec HP $\times 1,5$.

4.7 Bonus vitesse

+20 % vitesse de déplacement par région contrôlée :

$$v_{\text{effectif}} = v_{\text{base}} \times (1 + 0,20 \times n_{\text{régions}})$$

Constante code : `RegionSpeedBonusPerRegion = 0.20`.

5. Réseau

5.1 Parcours multijoueur



- Auth : email/mot de passe ou invité (device ID persistant).
- Matchmaking : 2–8 joueurs (`NakamaService.StartMatchmakingAsync`).
- Lobby in-match : countdown géré par le serveur relay (voir § 5.4).
- 1 camp par joueur humain ; camps restants neutres.

Côté client, les commandes passent par `NetworkCommandRouter` ; côté serveur, le module TypeScript `supkonquest_relay` valide et relaie les messages (voir § 5.2–5.7). Source : `supkonquest-server/NAKAMA_SERVER_TS.md`.

5.2 Module relay `supkonquest_relay`

Le runtime Nakama charge le module compilé `build/index.js` (via `npm run build` dans `supkonquest-server/`). Rôle : auth/matchmaking Nakama natifs + **relai de commandes** (pas de simulation complète du RTS côté serveur).

Fichier	Rôle
<code>main.ts</code>	Point d'entrée <code>InitModule</code> : enregistrement RPC, match, matchmaker
<code>messages.ts</code>	Constantes, opcodes, types, validation JSON
<code>lobby.ts</code>	Countdown lobby côté serveur, broadcast <code>MatchStart</code>
<code>match_handler.ts</code>	Handlers match + validation/relay des commandes gameplay
<code>match_rpc.ts</code>	RPC <code>supkonquest.find_match</code> (stub matchmaker)

Principes du relay :

- JSON camelCase, payloads simples, compatibles avec `NakamaService.cs` et `NetworkCommandRouter.cs`.
- Le serveur **valide** chaque commande (format, enveloppe, séquence) puis **relaye** aux autres clients.
- Le mode solo/offline reste indépendant : aucune dépendance Nakama en local.

5.3 Matchmaking

- Le client appelle `AddMatchmakerAsync` avec `properties.game = supkonquest` et `properties.mode = relay`.
- À l'appariement, le serveur crée un match : `nk.matchCreate('supkonquest_relay', { maxPlayers: '8' })`.
- Les clients rejoignent via `JoinMatchAsync`.
- Le lobby démarre dès **2 joueurs** connectés dans le match.

Constantes serveur (`messages.ts`) : minimum 2 joueurs, maximum 8, match idle fermé après 30 s sans personne connectée.

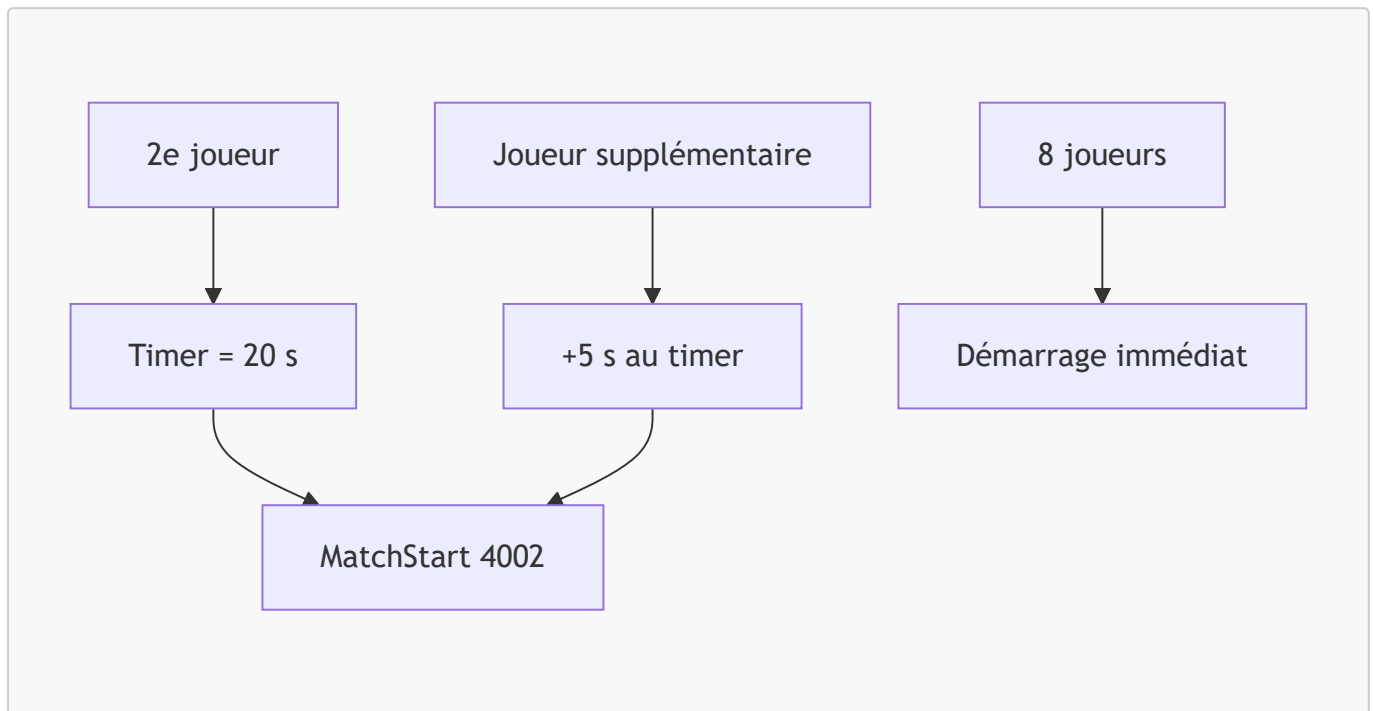
5.4 Opcodes lobby

Messages **serveur** → **clients uniquement** pendant la salle d'attente in-match.

Opcode	Nom	Payload JSON (camelCase)
4001	LobbyTick	<code>{ secondsRemaining, playerCount }</code> — environ 1/s pendant le lobby

Opcode	Nom	Payload JSON (camelCase)
4002	MatchStart	{ seed, orderedUserIds, matchId?, mapType } — orderedUserIds trié par ordre lexicographique ; mapType : 0 = Irridium, 1 = Alabasta, 2 = Torskey

Règles du countdown (lobby.ts) :



- **20 s** à l'arrivée du 2e joueur (pas de bonus pour le 2e).
- **+5 s** au temps restant à chaque joueur supplémentaire (3e, 4e, ...).
- Démarrage **immédiat** si **8** joueurs sont présents.
- Les commandes gameplay sont **rejetées** tant que **matchStarted** est false.
- **Seed** : hash déterministe du **matchId** (même algorithme que `NakamaService.GenerateSeedFromMatchId`).
- **mapType** : tiré au sort côté serveur à la création du match (0, 1 ou 2).
- **Équipes** : **userId** → **teamId** figé au **MatchStart** (index dans **orderedUserIds** + 1).

5.5 Opcodes gameplay relay

Messages **clients** ↔ **serveur** après le début de partie. Chaque payload inclut l'enveloppe (§ 5.7) plus les champs métier listés.

Opcode	Nom	Champs validés côté serveur
1001	BuyUnit	campId, teamId, unitType
1002	BuyShip	campId, teamId, shipType (Transport)
1003	SpawnShip	networkId, shipType, teamId, position, health
1004	BuildPort	campId, teamId, position, rotation, flipH
1005	SpawnUnit	networkId, unitType, teamId, campId, position, health
2001	MoveUnits	unitIds, start, target

Opcode	Nom	Champs validés côté serveur
2002	AttackCamp	unitIds, campId
2003	CampCaptured	campId, newTeamId
2004	MoveShips	shipIds, targetX/Y (sans position de départ)
2005	CampDamage	campId, damage, attackerTeamId
2006	BoardTransport	shipNetworkId, unitNetworkId, unitType, teamId, health
2007	TransportUnloaded	shipNetworkId + tableaux unités (ids, types, positions, pv)
2008	UnitsMoveToTransport	shipNetworkId, unitIds
6001	CastUltimate	unitIds, abilityId, targetX, targetY
6002	UltimateVfx	effet visuel d'ultime (relay client)
7001	UnitDamage	targetNetworkId, damage, attackerTeamId
7002	ShipDamage	targetNetworkId, damage, attackerTeamId
7003	EntityDied	networkId

Le client applique localement les messages reçus via `NetworkCommandRouter` ; le serveur ne recalcule pas le gameplay.

5.6 Opcodes déconnexion et reconnexion

Messages **serveur** → **clients** (broadcast système).

Opcode	Nom	Payload JSON
5002	PlayerLeaveCleanup	{ senderUserId: "server", sequence: 0, teamId }

Comportement actuel (`match_handler.ts`) :

- Au `matchLeave` après démarrage : le serveur diffuse **une seule fois 5002** par équipe (`cleanupAppliedByTeam`), puis le client exécute `GameManager.ApplyPlayerLeaveCleanup()` (unités/navires supprimés, camps neutralisés).

5.7 Enveloppe et validation serveur

Chaque commande client doit contenir une **enveloppe** dans le JSON :

```
{
  "senderUserId": "<userId Nakama>",
  "sequence": 42,
  "...": "champs métier"
}
```

Règles (`validateRelayEnvelope` dans `match_handler.ts`) :

- `senderUserId` doit correspondre à l'expéditeur réel du message.
- `sequence` doit être **strictement croissante** : `sequence == lastSequence + 1` par joueur.
- Commande rejetée si JSON invalide, opcode inconnu, match non démarré, ou validation métier échouée.
- Les broadcasts serveur système utilisent `senderUserId = "server"` et `sequence = 0`.

5.8 Déterminisme et simulation locale

- Même **seed** + même **mapType** → même carte, mêmes camps, même shuffle initial sur tous les clients.
- L'**économie** est calculée localement ; l'or peut être resynchronisé via l'opcode `3001` (GoldSnapshot).
- **Camps neutres** : simulés sur **chaque machine** (`CampSimple.IsLocallyOwned`) — tous les clients exécutent la même logique pour les défenseurs neutres, sans synchroniser chaque détail via le serveur.

Limite du modèle relay : le serveur ne valide pas la cohérence gameplay (or, PV, positions). Toute divergence entre clients doit être corrigée par des snapshots explicites ou une évolution vers un serveur autoritaire.

6. Écarts au cahier des charges — choix et justifications

Cette section documente les exigences **partiellement implémentées** ou **manquantes** identifiées, avec la raison du choix.

6.1 Partiellement implémentées

Camps neutres plus difficiles avec unités avancées

Exigence : les camps neutres devraient être plus difficiles et offrir des unités avancées.

Implémenté : 4 défenseurs (Infantry, Support, Heal, Range) avec $HP \times 1,5$. Heal et Range sont des unités tier 2.

Non implémenté : AntiArmor, Heavy, Mortar, Tank spawnable après capture du camp neutre.

Justification : plutôt que de placer des unités lourdes (AntiArmor, Heavy, Mortar, Tank) sur les camps neutres, la progression vers les unités avancées passe par le **système de tiers** (voir § 4.2) : investissement économique (tier 2) puis contrôle territorial (tier 3). Les neutres restent un défi early-game prévisible (4 types, $HP \times 1,5$) sans casser la courbe de progression ni l'équilibre économique.

Camps gardés par au moins une troupe en permanence

Exigence : un camp possédé doit toujours avoir au moins une unité vivante.

Implémenté : 4 défenseurs au spawn (ou après neutralisation).

Non implémenté : règle empêchant un camp possédé de se retrouver sans unité vivante.

Justification : en RTS classique, un camp vide est vulnérable mais capturable — cela récompense les raids et les feintes. Cela force les joueurs à gérer correctement leurs défenses afin d'éviter ces raids surprises. Imposer une garnison permanente n'apporte pas de profondeur stratégique. La tourelle du camp (5 dégâts/s, 600 px) offre déjà une défense passive.

IA stratégique avec personnalités distinctes

Exigence : IA plus ou moins optimisée avec personnalités variées.

Implémenté : Utility AI à 3 niveaux (Easy/Medium/Hard) avec compositions, naval, défense réactive, erreurs simulées.

Non implémenté : personnalités IA distinctes (agressif, économique, naval-only, etc.).

Justification : trois paliers de difficulté avec paramètres numériques différents offrent déjà une courbe de challenge. Des personnalités multiples multiplieraient les matrices de test sans gain proportionnel pour un projet académique. L'architecture Utility AI permet d'ajouter des poids de personnalité ultérieurement.

6.2 Manquantes

Mort mutuelle → camp neutre

Exigence : si le dernier défenseur et l'attaquant meurent simultanément, le camp redevient neutre.

État : le booléen `mutualKill` est calculé dans `Unit.Combat.Die()` et transmis à `OnDefenderDied()`, mais **non exploité car changement vers capture en deux phases**.

Justification du report : cas edge rare en pratique (timing exact de mort simultanée). La capture en deux phases (défenseurs puis bâtiment) rend le scénario impossible.

Les écarts documentés en § 6 sont des **choix de design assumés**, pas des bugs oubliés. Chaque entrée indique ce qui est implémenté, ce qui manque, et pourquoi.

7. Module reference (extrait)

Fichier	Rôle
<code>Scripts/Economy/GameManager.cs</code>	Économie, tiers, bonus région, assignation camps
<code>Scripts/Economy/VictoryManager.cs</code>	Condition et écran de victoire
<code>Scripts/Network/GameState.cs</code>	Session, seed, mode, carte
<code>Scripts/Network/NakamaService.cs</code>	Auth, matchmaking, lobby, relay
<code>Scripts/Network/NetworkCommandRouter.cs</code>	Enveloppes relay, application locale
<code>supkonquest-server/main.ts</code>	Module Nakama <code>supkonquest_relay</code>

Fichier	Rôle
<code>supkonquest-server/match_handler.ts</code>	Validation et relay des commandes
<code>supkonquest-server/lobby.ts</code>	Lobby countdown et MatchStart
<code>Scripts/Map/MapGenerator.cs</code>	Carte, navmesh, placement camps
<code>Scripts/Map/TerritoryConnectivity.cs</code>	Graphe de régions
<code>Scripts/Camps/CampSimple.*.cs</code>	Camp : prod, défense, naval, visuels
<code>Scripts/Units/Unit.*.cs</code>	Unité : combat, mouvement, heal, transport
<code>Scripts/Ships/Ship.*.cs</code>	Navire : combat, mouvement, transport
<code>Scripts/AI/AIController.cs</code>	IA Utility Easy/Medium/Hard
<code>Scripts/Selection/SelectionManager.cs</code>	Sélection souris, ordres
<code>Scripts/UI/GameHUD.cs</code>	HUD, achats, leaderboard
<code>Scripts/UI/KeybindingsManager.cs</code>	Macros sélection, ultimes

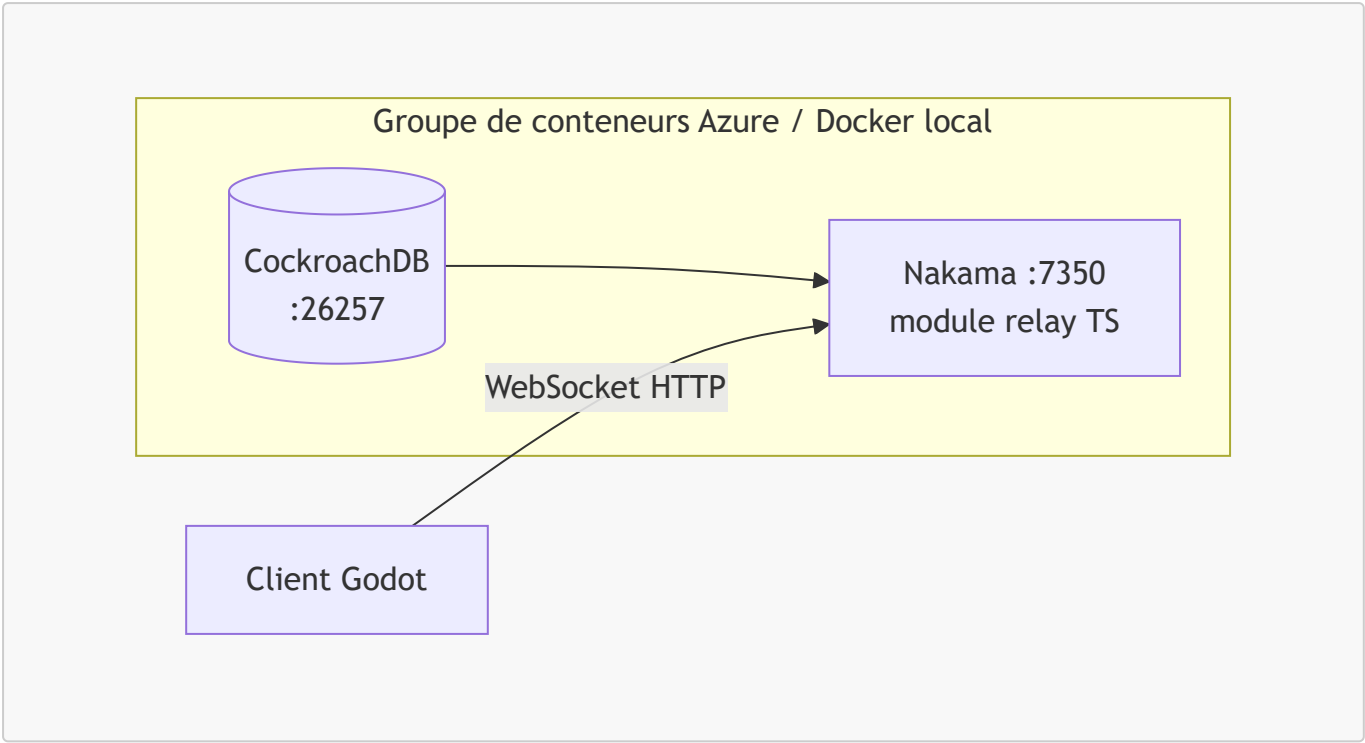
8. Parcours de lecture recommandé

- ☐ `README.md` — vue produit et promesses fonctionnelles
- ☐ `Scripts/Network/GameState.cs` — cycle de vie des parties
- ☐ `Scripts/Map/MapGenerator.cs` — apparition carte et camps
- ☐ `Scripts/Economy/GameManager.cs` — économie et tiers
- ☐ `Scripts/Camps/CampSimple.cs` + partials — capture, prod, défense
- ☐ `Scripts/Network/NakamaService.cs` — flux online
- ☐ `supkonquest-server/NAKAMA_SERVER_TS.md` — contrat relay (résumé intégré en § 5)

9. Annexe — Serveur Nakama (développeurs)

Section réservée aux développeurs qui modifient le client Godot ou le module relay TypeScript. Les joueurs se connectent aux serveurs en ligne ; aucune installation locale n'est requise pour jouer.

Architecture



Le backend repose sur **deux conteneurs** (définis dans `supkonquest-server/docker-compose.yml`, format Azure Container Instances) :

Conteneur	Image	Rôle
cockroachdb	cockroachdb/cockroach:latest-v23.1	Base de données.
nakama	heroiclabs/nakama:3.22.0 (prod ACI) ou image custom via Dockerfile (local)	API jeu, auth, matchmaking, relay. Charge le module TS compilé (<code>build/index.js</code>).

Ports exposés :

Port	Service
26257	CockroachDB (SQL)
8080	Console admin CockroachDB
7349	Nakama gRPC
7350	Nakama HTTP / API client (port utilisé par le jeu)
7351	Nakama console

Au démarrage, Nakama exécute les migrations puis lance le serveur :

```
/nakama/nakama migrate up --database.address root@...:26257
/nakama/nakama --database.address root@...:26257 --logger.level DEBUG --session.token_expiry_sec 7200
```

En production Azure, les deux conteneurs partagent le réseau du groupe (`127.0.0.1:26257`). Le module relay est monté via un volume Azure Files sur `/nakama/data/modules/`.

Prérequis locaux

- Docker Desktop
- Node.js + npm (compilation du module TS)

Démarrage local (Docker Compose)

Le fichier `docker-compose.yml` du dépôt serveur est au **format Azure ACI**, pas au format Compose classique. Utilisez `docker-compose.local.yml` pour le développement local (voir ci-dessous).

1. Compiler le module TypeScript

```
cd supkonquest-server  
npm install  
npm run build
```

Vérifiez que `build/index.js` existe.

2. Lancer CockroachDB + Nakama

Créez un fichier `docker-compose.local.yml` (ou lancez directement) avec cette structure — calquée sur le compose Azure :

```
services:
  cockroachdb:
    image: cockroachdb/cockroach:latest-v23.1
    command: start-single-node --insecure --store=attrs=ssd,path=/var/lib/cockroach/
    ports:
      - "26257:26257"
      - "8080:8080"

  nakama:
    build: .
    depends_on:
      - cockroachdb
    entrypoint:
      - "/bin/sh"
      - "-ecx"
      - >
        /nakama/nakama migrate up --database.address root@cockroachdb:26257 &&
        exec /nakama/nakama --config /nakama/data/local.yml
        --name nakama1 --database.address root@cockroachdb:26257
        --logger.level DEBUG --session.token_expiry_sec 7200
    ports:
      - "7349:7349"
      - "7350:7350"
      - "7351:7351"
```

```
docker compose -f docker-compose.local.yml up --build
```

Le **Dockerfile** embarque **build/index.js** et **local.yml** dans l'image Nakama. En local, recompilez (**npm run build**) puis relancez **docker compose up --build** après chaque modification du module TS.

3. Vérifier

- Console CockroachDB : <http://localhost:8080>
- Nakama prêt quand les logs affichent le démarrage sans erreur de migration.

Déploiement Azure (production)

Le fichier **docker-compose.yml** du dépôt serveur décrit un **groupe de conteneurs Azure** (**api-version: 2019-12-01**) :

- Conteneur **cockroachdb** + conteneur **nakama** dans le même groupe réseau.
- Volume **Azure Files** (**nakamamodules**) monté sur **/nakama/data/modules/** pour le module relay compilé.
- IP publique sur les ports 7350, 7351 et 8080.

Déployer via Azure CLI ou le portail à partir de **docker-compose.yml** ou **nakama-deploy.yml**.

Configuration client Godot

Dans `project.godot`, section `[nakama]` :

Paramètre	Local	Production
host	127.0.0.1	IP ou domaine du serveur Azure
port	7350	7350
server_key	defaultkey	defaultkey (ou clé configurée côté serveur)

Redémarrer Godot après modification.

Test multijoueur local (2 clients)

Lancez **deux instances** du jeu avec un slot Nakama différent chacune (fichiers `user://nakama_device_id_1.txt`, `user://nakama_auth_session_1.dat`, etc.).

PowerShell (Windows) :

```
& "C:\Chemin\Vers\Godot_v4.5.1-stable_mono_win64.exe" --path "C:\Chemin\Vers\SupKonQuest" --nakama-slot=1  
& "C:\Chemin\Vers\Godot_v4.5.1-stable_mono_win64.exe" --path "C:\Chemin\Vers\SupKonQuest" --nakama-slot=2
```

Remplacez les chemins par votre exécutable Godot **.NET (mono)** et la racine du projet. Utilisez la variante `mono` de Godot 4.5 pour le support C#.

Chaque fenêtre simule un joueur distinct : connectez-vous (invité ou compte) sur les deux, puis lancez le matchmaking depuis l'une d'elles.

Arrêt et nettoyage

```
docker compose -f docker-compose.local.yml down  
# Supprimer aussi les volumes si besoin d'une base vierge :  
docker compose -f docker-compose.local.yml down -v
```

Le **contrat relay** (opcodes, lobby, validation) est documenté en § 5. Le fichier source `supkonquest-server/NAKAMA_SERVER_TS.md` reste la référence courte côté dépôt serveur.